

Name:P.Varshitha

Batch:06

Hall Tno. :2403A510G8

Lab Test:04

Q1. A company wants dynamic monthly financial reports.

a) Create schema for multi-table reporting.

Prompt: Write code to Create schema for multi-table reporting fro monthly financial reports

Code:

```
task01a.py > ...
1  # financial_report.py
2  # 100% self-contained: uses SQLite (built-in).
3  # No external database or installation required.
4
5  import sqlite3
6  from datetime import datetime
7
8  # Connect to an in-memory SQLite DB (disappears when program ends)
9  conn = sqlite3.connect(":memory:")
10 cur = conn.cursor()
11
12 # -----
13 # CREATE SCHEMA
14 # -----
15 cur.executescript("""
16 CREATE TABLE companies (
17     company_id INTEGER PRIMARY KEY,
18     name TEXT
19 );
20
21 CREATE TABLE invoices (
22     invoice_id INTEGER PRIMARY KEY,
23     company_id INTEGER,
24     issue_date TEXT,
25     FOREIGN KEY(company_id) REFERENCES companies(company_id)
26 );
27
28 CREATE TABLE invoice_items (
29     item_id INTEGER PRIMARY KEY,
30     invoice_id INTEGER,
31     quantity REAL,
32     unit_price REAL,
33     FOREIGN KEY(invoice_id) REFERENCES invoices(invoice_id)
34 );
35
36 INSERT INTO companies (name) VALUES ('Acme Corp');
37 """)
38
39 # -----
40 # INSERT SAMPLE DATA
```

```

42 # Revenue inserted for: Jan, Feb, Mar, May, Jun
43 cur.executescript("""
44 -- JAN
45 INSERT INTO invoices VALUES (1, 1, '2025-01-15');
46 INSERT INTO invoice_items VALUES (1, 1, 10, 25); -- 250
47 INSERT INTO invoice_items VALUES (2, 1, 5, 100); -- 500 (total Jan = 750)
48
49 -- FEB
50 INSERT INTO invoices VALUES (2, 1, '2025-02-10');
51 INSERT INTO invoice_items VALUES (3, 2, 8, 25); -- 200
52 INSERT INTO invoice_items VALUES (4, 2, 2, 120); -- 240 (total Feb = 440)
53
54 -- MAR
55 INSERT INTO invoices VALUES (3, 1, '2025-03-05');
56 INSERT INTO invoice_items VALUES (5, 3, 2, 25); -- 50
57 INSERT INTO invoice_items VALUES (6, 3, 1, 150); -- 150 (total Mar = 200)
58
59 -- MAY
60 INSERT INTO invoices VALUES (4, 1, '2025-05-02');
61 INSERT INTO invoice_items VALUES (7, 4, 20, 50); -- 1000
62 INSERT INTO invoice_items VALUES (8, 4, 10, 120); -- 1200 (May = 2200)
63
64 -- JUN
65 INSERT INTO invoices VALUES (5, 1, '2025-06-10');
66 INSERT INTO invoice_items VALUES (9, 5, 5, 25); -- 125 (June = 125)
67 """)
68
69 # -----
70 # QUERY: Month-wise revenue + % growth
71 # -----
72 query = """
73 WITH monthly AS (
74     SELECT
75         strftime('%Y-%m', issue_date) AS month,
76         SUM(quantity * unit_price) AS revenue
77     FROM invoices
78     JOIN invoice_items USING(invoice_id)
79     GROUP BY strftime('%Y-%m', issue_date)
80 )
81 SELECT
82     month,
83     revenue,

```

```

81 SELECT
82     month,
83     revenue,
84     LAG(revenue) OVER (ORDER BY month) AS prev_revenue,
85     CASE
86         WHEN LAG(revenue) OVER (ORDER BY month) IS NULL THEN NULL
87         WHEN LAG(revenue) OVER (ORDER BY month) = 0 THEN NULL
88         ELSE ROUND((revenue - LAG(revenue) OVER (ORDER BY month))
89             * 100.0 / LAG(revenue) OVER (ORDER BY month), 2)
90     END AS pct_growth
91 FROM monthly
92 ORDER BY month;
93 """
94
95 # Enable window functions (SQLite supports them)
96 cur.execute(query)
97 rows = cur.fetchall()
98
99 # -----
100 # PRINT OUTPUT
101 # -----
102 print(f"{'Month':<10}{'Revenue':<12}{'Prev Revenue':<15}{'% Growth':<10}")
103 for r in rows:
104     print(f"{r[0]:<10}{r[1]:<12}{str(r[2]):<15}{str(r[3])}")
105
106 conn.close()
107

```

Output:(of the schemas with data)

```

python -u "d:\LABS\Ai Assistant\lab exam codes\lab exam 04\task01a.py"

```

Month	Revenue	Prev Revenue	% Growth
2025-01	750.0	None	None
2025-02	440.0	750.0	-41.33
2025-03	200.0	440.0	-54.55
2025-05	2200.0	200.0	1000.0
2025-06	125.0	2200.0	-94.32

```

❖ PS D:\LABS\Ai Assistant\lab exam codes\lab exam 04>

```

Observation:

This schema uses basic SQLite tables to organize companies, customers, products, invoices, and invoice items in a clean, connected structure. It keeps all financial data separated but linked through foreign keys, making it easy to calculate monthly revenue and build any type of financial report. Because it uses SQLite (built into Python), the whole setup runs directly in VS Code without needing any external database.

b) Write AI-assisted SQL for month-wise revenue + percentage growth

Prompt: Write SQL program that calculate month-wise revenue + percentage growth form previous table

Code:

```
task1b.py > ...
1  # q1b final.py
2  # Month-wise revenue + percentage growth using SQLite
3  # Output only: Month, Revenue, Growth %
4
5  import sqlite3
6
7  conn = sqlite3.connect(":memory:")
8  cur = conn.cursor()
9
10 # Create tables
11 cur.executescript("""
12 CREATE TABLE invoices (
13     invoice_id INTEGER PRIMARY KEY,
14     issue_date TEXT
15 );
16 CREATE TABLE invoice_items (
17     item_id INTEGER PRIMARY KEY,
18     invoice_id INTEGER,
19     quantity REAL,
20     unit_price REAL,
21     FOREIGN KEY(invoice_id) REFERENCES invoices(invoice_id)
22 );
23 """)
24
25 # Insert data
26 cur.executescript("""
27 INSERT INTO invoices VALUES (1, '2025-01-15');
28 INSERT INTO invoice_items VALUES (1, 1, 10, 25);
29 INSERT INTO invoice_items VALUES (2, 1, 5, 100);
30
31 INSERT INTO invoices VALUES (2, '2025-02-10');
32 INSERT INTO invoice_items VALUES (3, 2, 8, 25);
33 INSERT INTO invoice_items VALUES (4, 2, 2, 120);
34
35 INSERT INTO invoices VALUES (3, '2025-03-05');
36 INSERT INTO invoice_items VALUES (5, 3, 2, 25);
37 INSERT INTO invoice_items VALUES (6, 3, 1, 150);
38
39 INSERT INTO invoices VALUES (4, '2025-05-02');
40 INSERT INTO invoice_items VALUES (7, 4, 20, 50);
41 INSERT INTO invoice_items VALUES (8, 4, 10, 120);
42 """)
```

```

43 INSERT INTO invoices VALUES (5, '2025-06-10');
44 INSERT INTO invoice_items VALUES (9, 5, 5, 25);
45 """
46
47 # Month-wise revenue + % growth
48 query = """
49 WITH monthly AS (
50     SELECT
51         strftime('%Y-%m', issue_date) AS month,
52         SUM(quantity * unit_price) AS revenue
53     FROM invoices
54     JOIN invoice_items USING(invoice_id)
55     GROUP BY strftime('%Y-%m', issue_date)
56 )
57 SELECT
58     month,
59     revenue,
60     CASE
61         WHEN LAG(revenue) OVER (ORDER BY month) IS NULL THEN NULL
62         WHEN LAG(revenue) OVER (ORDER BY month) = 0 THEN NULL
63         ELSE ROUND((revenue - LAG(revenue) OVER (ORDER BY month))
64                   * 100.0 / LAG(revenue) OVER (ORDER BY month), 2)
65     END AS pct_growth
66 FROM monthly
67 ORDER BY month;
68 """
69
70 cur.execute(query)
71 rows = cur.fetchall()
72
73 # Final simple output (only what is needed)
74 print("Month      Revenue      % Growth")
75 for r in rows:
76     print(f"{r[0]:<10}{r[1]:<12}{str(r[2])}")
77
78 conn.close()

```

OutPut:

```

PS D:\LABs\Ai Assistant\lab exam codes\lab exam 04> python -u "d:\LABs\Ai Assistant\lab exam codes\lab exam 04\task1b.py"
Month      Revenue      % Growth
2025-01     750.0         None
2025-02     440.0        -41.33
2025-03     200.0        -54.55
2025-05    2200.0       1000.0
2025-06     125.0        -94.32

```

Observation:

This SQL calculates each month's revenue by summing quantity × price from invoice items. It then uses SQLite's LAG() function to compare the revenue with the previous month and compute the percentage growth. The output clearly shows how revenue changes from month to month.

Q2. AI suggests automated report generation using Python-Pandas.

a) Write pseudocode.

Prompt:.

Code:

```
/*  
START  
  
Load dataset into a DataFrame  
Check for missing or incorrect values  
Clean the data by removing or fixing invalid entries  
  
Create a new column:  
    revenue = quantity × price  
  
Group data by month  
Calculate total revenue for each month  
  
Store results in a new report DataFrame  
Display or export the final monthly report  
  
END  
*/
```

```

task02.py > ...
1  # q2a_pseudocode_to_code.py
2  # Implementation of the pseudocode using Python-Pandas
3
4  import pandas as pd
5
6  # Sample dataset (no file needed)
7  data = {
8      "month": ["Jan", "Jan", "Feb", "Feb", "Mar"],
9      "quantity": [10, 5, 8, 2, 4],
10     "price": [20, 30, 20, 25, 40]
11 }
12
13 # Load dataset into DataFrame
14 df = pd.DataFrame(data)
15
16 # Check and clean missing values
17 df = df.dropna()
18
19 # Create revenue column
20 df["revenue"] = df["quantity"] * df["price"]
21
22 # Group by month and calculate total revenue
23 report = df.groupby("month")["revenue"].sum().reset_index()
24
25 # Display final monthly report
26 print("Monthly Revenue Report:")
27 print(report)
28

```

Output:

```

Monthly Revenue Report:
  month  revenue
0  Feb      210
1  Jan      350
2  Mar      160
❖ PS D:\LABs\Ai Assistant\lab exam codes\lab exam 04> 

```

Observation:

This code follows the pseudocode exactly by loading data, cleaning it, calculating revenue, and grouping it by month. Pandas handles all operations efficiently, making automated report generation simple. The final output shows a clean monthly revenue summary.

b) Explain how to ensure accuracy.

Prompt:(if needed) (Expalin how to ensure Accuracy while working with python-pandas.)

To ensure accuracy in automated Pandas reports we should to following things:

1. **Clean and validate the dataset** – remove duplicates, handle missing values, and fix incorrect formats before calculations.
2. **Convert all data types correctly** – especially dates and numerical columns so calculations behave correctly.
3. **Use consistency checks** – such as comparing totals, checking summary statistics, or verifying sample rows manually.
4. **Apply standardized formulas** – ensuring revenue or growth calculations use the same logic every time.
5. **Automate error detection** – flagging abnormal values, negative quantities, or unrealistic spikes.